

Практика 2. Рекурсия в λ -исчислении

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ Синтаксис и семантика λ -исчисления
- ▶ Абстракция по данным в λ -исчислении: кортежи, натуральные числа, логические величины
- + Нормальные формы
- + Рекурсия и **Y**-комбинатор
- + Стратегии редукции

Содержание

1. Ещё метаинформация о курсе
2. Примитивная рекурсия
3. Рекурсия и комбинаторы неподвижной точки
4. Кодирование данных в лямбда-исчислении
 - ▶ Значения-варианты
 - ▶ Списки
5. Энергичность и ленивость
6. Материалы (☼)

Содержание

1. Ещё метаинформация о курсе
2. Примитивная рекурсия
3. Рекурсия и комбинаторы неподвижной точки
4. Кодирование данных в лямбда-исчислении
 - ▶ Значения-варианты
 - ▶ Списки
5. Энергичность и ленивость
6. Материалы (☺)

Зачем изучать ФП/теорию языков программирования

Для самоопределения

- ▶ ФП — лакмусовая бумажка: понравился курс $\Rightarrow$ скорее всего понравится профессионально заниматься языками, в науке или промышленности

Чтобы быть более хорошими программистами

- ▶ Узнать, что вообще в принципе идейно бывает в языках
 1. Проще изучать новые языки
 2. Придумываем решение в пространстве возможного $\Rightarrow$ А потом аппроксимируем в терминах конкретного языка
- ▶ Дисциплинирует ум и учит выбирать более лаконичные и композируемые абстракции
- ▶ Прививает хорошие практики программирования и аккуратное отношение к эффектам

Чтобы иметь базу для дальнейшего погружения в науку о языках программирования

- ▶ ФП на пересечении многих областей (вычислимость, доказательства, категории...)
- ▶ ФП — неотъемлемая часть академической традиции теории языков программирования
- ▶ Proof engineering и машинно-проверяемые доказательства — всё тут

Тutorial по прохождению

- ▶ Работайте с эмоциями и держитесь рациональных решений
- ▶ Старайтесь спокойно относиться к неудачам и работать с их причинами
 - ▶ Тайм-менеджмент и менеджмент ресурсов
 - ▶ Садитесь за домашки не в последний момент
 - ▶ Старайтесь понять, а не “натыкать” задание
- ▶ Не стесняйтесь просить помощи у преподавателей и одногруппников
 - ▶ Иногда можно организовывать дополнительную встречу, если с темой у многих проблемы
- ▶ Повторяйте материал перед парами
 - ▶ Времени требуется не много, но толку — куча!
 - ▶ Для практик более-менее достаточно повторить последнюю лекцию
- ▶ Спите! Иначе всё более-менее без толку
- ▶ Отдыхайте! Иначе на дистанции вы будете сильно терять в производительности

Полезная [памятка](#) от кураторов Computer Science Center.

Содержание

1. Ещё метаинформация о курсе
2. Примитивная рекурсия
3. Рекурсия и комбинаторы неподвижной точки
4. Кодирование данных в лямбда-исчислении
 - ▶ Значения-варианты
 - ▶ Списки
5. Энергичность и ленивость
6. Материалы (☀)

Цикл по заданному интервалу

- ▶ Любой цикл по интервалу можно записать в следующем виде:

```
def rec(f, ini, n):
    state = ini
    for i in range(0, n):
        state = f(i, state)
    return state
```

- ▶ Узнаем общий вид конечного результата с помощью вызова:

```
rec(f = lambda i, state: "f(" + str(i) + ", " + state + ")",
    ini = "z", n = 5)
> "f(4, f(3, f(2, f(1, f(0, z)))))"
```

todo На что похоже?

todo А что если мы хотим обновлять несколько переменных в цикле?



На числа Чёрча!



Ну будем поддерживать в state кортеж

Примитивная рекурсия в λ -исчислении

Примитивная рекурсия

$$\mathbf{rec} \ f \ \mathit{ini} \ n =_{\beta} f \ (n - 1) \ (f \ (n - 2) \ (\dots \ (f \ 1 \ (f \ 0 \ \mathit{ini})) \dots))$$

- ▶ Хотим выразить в виде $q \ (s \ (s \ \dots (s \ z) \dots))$, какими будут q , s и z ?
- ▶ Из итерации в итерацию будем передавать пару: номер итерации; результат предыдущих
- ▶ Начальное состояние — **start** $\mathit{ini} := \mathbf{pair} \ 0 \ \mathit{ini}$
- ▶ Одна итерация:¹

$$\mathbf{step} \ f \ (\mathbf{pair} \ i \ res) := \mathbf{pair} \ (\mathbf{suc} \ i) \ (f \ i \ res)$$

- ▶ Итерирование

$$\mathbf{iter} \ f \ \mathit{ini} \ n := \mathbf{natElim} \ n \ (\mathbf{step} \ f) \ (\mathbf{start} \ \mathit{ini})$$

- ▶ И наконец

$$\mathbf{rec} \ f \ \mathit{ini} \ n := \mathbf{snd} \ (\mathbf{iter} \ f \ \mathit{ini} \ n)$$

¹Пусть сокращение **pair** $i \ res$ слева значит: принимаем пару p , вместо i подразумеваем **fst** p ; res — **snd** p .

Примеры применения примитивной рекурсии

Примитивная рекурсия

$$\mathbf{rec} \ f \ ini \ n =_{\beta} f \ (n - 1) \ (f \ (n - 2) \ (\dots \ (f \ 1 \ (f \ 0 \ ini)) \dots))$$

todo Реализуйте функцию суммирования чисел от 1 до n

todo Реализуйте факториал

todo Реализуйте **pred**

⚡ **sum** $n := \mathbf{rec} \ (\lambda i \ res. \mathbf{plus} \ (\mathbf{suc} \ i) \ res) \ \mathbf{0} \ n$

⚡ **fac** $n := \mathbf{rec} \ (\lambda i \ res. \mathbf{mult} \ (\mathbf{suc} \ i) \ res) \ \mathbf{1} \ n$

⚡ **pred** $n := \mathbf{rec} \ \mathbf{K} \ \mathbf{0} \ n$

► Во всех случаях можно сделать η -редукцию

Проверка факториала

- $$\begin{aligned} \text{fac } 2 &\rightsquigarrow_{\beta} \text{snd } (2 \text{ (step } (\lambda i \text{ r. mult (suc i) r)) (pair 0 1))} \rightsquigarrow_{\beta} \\ &\text{snd (step } (\lambda i \text{ r. mult (suc i) r) ((\text{step } (\lambda i \text{ r. mult (suc i) r)) (pair 0 1)))} \rightsquigarrow_{\beta} \\ &\text{snd (step } (\lambda i \text{ r. mult (suc i) r) (pair (suc 0) ((\lambda i \text{ r. mult (suc i) r) 0 1)))} \rightsquigarrow_{\beta} \\ &\text{snd (step } (\lambda i \text{ r. mult (suc i) r) (pair 1 (mult (suc 0) 1)))} \rightsquigarrow_{\beta} \\ &\text{snd (step } (\lambda i \text{ r. mult (suc i) r) (pair 1 1))} \rightsquigarrow_{\beta} \\ &\text{snd (pair (suc 1) ((\lambda i \text{ r. mult (suc i) r) 1 1))} \rightsquigarrow_{\beta} (\lambda i \text{ r. mult (suc i) r) 1 1} \rightsquigarrow_{\beta} \\ &\text{mult (suc 1) 1} \rightsquigarrow_{\beta} 2 \end{aligned}$$
- Да ну серьёзно??
- Через свойство примитивной рекурсии сильно проще

$$\begin{aligned} \text{fac } 3 &\rightsquigarrow_{\beta} f \text{ 2 (f 1 (f 0 1))} \\ &\rightsquigarrow_{\beta} (\lambda i \text{ res. mult (suc i) res) 2 (f 1 (f 0 1))} \\ &\rightsquigarrow_{\beta} \text{mult (suc 2) (f 1 (f 0 1))} \\ &\rightsquigarrow_{\beta} \text{mult 3 (mult 2 (mult 1 1))} \rightsquigarrow_{\beta} 6 \end{aligned}$$

- Важно грамотно выбирать уровень абстракции

Содержание

1. Ещё метаинформация о курсе
2. Примитивная рекурсия
3. Рекурсия и комбинаторы неподвижной точки
4. Кодирование данных в лямбда-исчислении
 - ▶ Значения-варианты
 - ▶ Списки
5. Энергичность и ленивость
6. Материалы (☺)

Рекурсия

- ▶ Одной примитивной рекурсии в языке недостаточно, чтобы уметь всё
- ▶ Чтобы добиться полноты, нужна полноценная рекурсия
- ▶ В прикладные языки рекурсия встроена — достаточно в теле именованной функции вызвать её саму
- ▶ Но у нас в языке нет именованных функций, же?!

Рекурсивные уравнения на термах

Хотим подобрать такой терм **fac**, чтобы $\mathbf{fac} \ N =_{\beta} \mathbf{if} \ (\mathbf{iszero} \ N) \ 1 \ (\mathbf{mult} \ N \ (\mathbf{fac} \ (\mathbf{pred} \ N)))$.

Решаем рекурсивное уравнение на термах!

1. Абстрагируемся в правой части по **fac** и N :

$$\mathbf{fac} \ N =_{\beta} \underbrace{(\lambda r \ n. \mathbf{if} \ (\mathbf{iszero} \ \boxed{n}) \ 1 \ (\mathbf{mult} \ \boxed{n} \ (\boxed{r} \ (\mathbf{pred} \ \boxed{n}))))}_{\mathbf{fac}'}} \ \mathbf{fac} \ N$$

2. Поскольку правые аппликанды совпадают ($N \equiv N$), рассмотрим левые части:

$$\mathbf{fac} =_{\beta} \mathbf{fac}' \ \mathbf{fac}$$

3. То есть **fac** — неподвижная точка отображения **fac'**! Найдём её **Y**-комбинатором:

$$\mathbf{fac} := \mathbf{Y} \ \mathbf{fac}' =_{\beta} \mathbf{Y} \ (\lambda r \ n. \mathbf{if} \ (\mathbf{iszero} \ n) \ 1 \ (\mathbf{mult} \ n \ (r \ (\mathbf{pred} \ n))))$$

4. Делаем проверку, зная, что $\mathbf{Y} \ \mathbf{fac}' =_{\beta} \mathbf{fac}' \ (\mathbf{Y} \ \mathbf{fac}')$:

$$\begin{aligned} \mathbf{fac} \ N &=_{\beta} \mathbf{Y} \ \mathbf{fac}' \ N =_{\beta} \mathbf{fac}' \ (\mathbf{Y} \ \mathbf{fac}') \ N \\ &=_{\beta} (\lambda r \ n. \mathbf{if} \ (\mathbf{iszero} \ \boxed{n}) \ 1 \ (\mathbf{mult} \ \boxed{n} \ (\boxed{r} \ (\mathbf{pred} \ \boxed{n})))) \ (\mathbf{Y} \ \mathbf{fac}') \ N \\ &=_{\beta} \mathbf{if} \ (\mathbf{iszero} \ \boxed{N}) \ 1 \ (\mathbf{mult} \ \boxed{N} \ (\boxed{\mathbf{Y} \ \mathbf{fac}'} \ (\mathbf{pred} \ \boxed{N}))) \end{aligned}$$

Больше рекурсивных функций!

Перепишите через комбинатор неподвижной точки (решите рекурсивные уравнения):

todo $\text{fib } n =_{\beta} \text{if } (\text{le } n \ 2) \ 1 \ (\text{plus } (\text{fib } (\text{minus } n \ 1)) \ (\text{fib } (\text{minus } n \ 2)))$

todo $F =_{\beta} M \ F$

 $\text{fib} := \mathbf{Y} \ \lambda r. \text{if } (\text{le } n \ 2) \ 1 \ (\text{plus } (r \ (\text{minus } n \ 1)) \ (r \ (\text{minus } n \ 2)))$

 $F =_{\beta} \mathbf{Y} \ M$

Комбинатор рекурсии (неподвижной точки)

Свойство комбинатора неподвижной точки (**Y** — пример такого комбинатора)

$$\underbrace{\mathbf{Y} F}_X =_\beta F (\underbrace{\mathbf{Y} F}_X)$$

$$\mathbf{fac} := \mathbf{Y} \underbrace{(\lambda \boxed{r} n. \text{if } (\mathbf{iszero} \ n) \ \mathbf{1} \ (\mathbf{mult} \ n \ (\boxed{r} \ (\mathbf{pred} \ n))))}_{\mathbf{fac}'}$$

todo Стоп, стоп. Как анонимная функция стала рекурсивной?

⚡ **Y** передаёт в нашу функцию терм, который ведёт себя ровно так же, как она!

Договорённость о рекурсивных синонимах

Вводим рекурсию как синтаксический сахар для определения выше:

$$\mathbf{fac} \ n := \text{if } (\mathbf{iszero} \ n) \ \mathbf{1} \ (\mathbf{mult} \ n \ (\boxed{\mathbf{fac}} \ (\mathbf{pred} \ n)))$$

Выбор инструмента: примитивная рекурсия vs рекурсия (⚙)

- ▶ Примитивная рекурсия
 - ▶ Задаёт процесс из конечного, заранее заданного, количества шагов
 - ▶ Аналог цикла `for in range` в современных языках
- ▶ Рекурсия
 - ▶ Количество шагов определяется в процессе (и никак иначе: halting problem)
 - ▶ Завершение процесса определяется выполнением произвольного условия (базы рекурсии)
 - ▶ В современных языках выражается рекурсией² и циклом `while condition: ...`
- ▶ Всегда нужно выбирать *наименее могущественный* из подходящих инструментов
 - ▶ Проще для рассуждений и доказательства свойств
 - ▶ Меньше пространство для ошибок
 - ▶ Скорее всего, код будет эффективнее исполняться
 - ▶ В частности, какие функции примитивно-рекурсивные, а какие — нет, рассказывает теория вычислимости

²Хвостовая рекурсия полностью эквивалентна обычной, хотя и похожа на примитивную (см. [trampolining](#)).

Содержание

1. Ещё метаинформация о курсе
2. Примитивная рекурсия
3. Рекурсия и комбинаторы неподвижной точки
4. Кодирование данных в лямбда-исчислении
 - ▶ Значения-варианты
 - ▶ Списки
5. Энергичность и ленивость
6. Материалы (☀)

Значения-варианты

Одна сущность может быть нескольких различных видов:

- ▶ Пользователь может быть авторизованный и не авторизованный
- ▶ Вершина программы может быть константой, а может — бинарной операцией

Хотим, чтобы в зависимости от её вида, сущность обрабатывал разный код:

```
def eval(node):  
    if is_const(node):  
        return node.value    # У константы должно быть значение  
    elif is_plus(node):  
        return eval(node.left) + eval(node.right)    # У плюса - операнды
```

Моделируем значения-варианты

API вариантов

- ▶ Один вариант будем называть “левым”, второй — “правым”
- ▶ Два конструктора **inl** и **inr**
- ▶ Элиминатор позволяет выполнить разный код в зависимости от варианта: **either**

Законы вариантов

1. $\forall f\ g\ x. \mathbf{either}\ f\ g\ (\mathbf{inl}\ x) =_{\beta} f\ x$
2. $\forall f\ g\ y. \mathbf{either}\ f\ g\ (\mathbf{inr}\ y) =_{\beta} g\ y$

Тегированное представление вариантов

Законы вариантов

1. $\forall f\ g\ x. \mathbf{either}\ f\ g\ (\mathbf{inl}\ x) =_{\beta} f\ x$
2. $\forall f\ g\ y. \mathbf{either}\ f\ g\ (\mathbf{inr}\ y) =_{\beta} g\ y$

Конструкторы

- ▶ Будем приписывать булев тег для различения вариантов
- ▶ $\mathbf{inl}\ x := \mathbf{pair\ true}\ x$
- ▶ $\mathbf{inr}\ y := \mathbf{pair\ false}\ y$

Элиминатор

- ▶ Вызвать нужную функцию в зависимости от тега
- ▶ $\mathbf{either}\ f\ g\ (\mathbf{pair}\ tag\ val) := \mathbf{if}\ tag\ (f\ val)\ (g\ val)$

Кодирование вариантов Чёрча

Законы вариантов

1. $\forall f\ g\ x. \mathbf{either}\ f\ g\ (\mathbf{inl}\ x) =_{\beta} f\ x$
2. $\forall f\ g\ y. \mathbf{either}\ f\ g\ (\mathbf{inr}\ y) =_{\beta} g\ y$

Конструкторы

- ▶ Принимают один аргумент
- ▶ Принимают два действия, из которых выполняют нужное на аргументе
- ▶ $\mathbf{inl}\ x := \lambda f\ g. f\ x$
- ▶ $\mathbf{inr}\ y := \lambda f\ g. g\ y$

Элиминатор

- ▶ Просто передаёт функции значению для обработки содержимого
- ▶ $\mathbf{either}\ f\ g\ v := v\ f\ g$

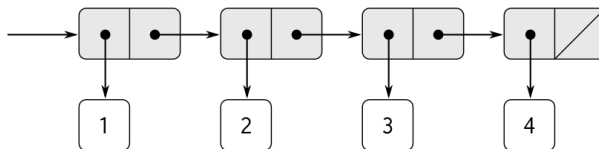
Содержание

1. Ещё метаинформация о курсе
2. Примитивная рекурсия
3. Рекурсия и комбинаторы неподвижной точки
4. Кодирование данных в лямбда-исчислении
 - ▶ Значения-варианты
 - ▶ Списки
5. Энергичность и ленивость
6. Материалы (☀)

Списки

Иногда требуется хранить много сущностей одновременно, и при этом:

- ▶ Уметь добавить в коллекцию новую сущность
- ▶ Найти сущность в коллекции по индексу



Моделируем список

API списков

- ▶ **nil** — пустой список
- ▶ **cons** x t — список из элемента x в голове и списка-хвоста t

e.g. $[1, 2, 3] = \text{cons } 1 (\text{cons } 2 (\text{cons } 3 \text{ nil}))$

- ▶ Элиминатор, аккумулирующий элементы: **fold**

e.g. $\text{fold } [1, 2, 3] f \text{ ini} =_{\beta} f \ 1 (f \ 2 (f \ 3 \text{ ini}))$

Законы списков

1. $\forall f \text{ ini}. \text{fold nil } f \text{ ini} =_{\beta} \text{ini}$
2. $\forall x \ t \ f \ \text{ini}. \text{fold (cons } x \ t) f \text{ ini} =_{\beta} f \ x (\text{fold } t \ f \ \text{ini})$

Списки на вариантах

Законы списков

1. $\forall f\ ini. \mathbf{fold\ nil}\ f\ ini =_{\beta} ini$
2. $\forall x\ t\ f\ ini. \mathbf{fold\ (cons\ x\ t)}\ f\ ini =_{\beta} f\ x\ (\mathbf{fold\ t\ f\ ini})$

Конструкторы

- ▶ Вершина либо пустая, либо пара из элемента и хвоста
- ▶ $\mathbf{nil} := \mathbf{inl\ 0}$ ¹
- ▶ $\mathbf{cons\ x\ t} := \mathbf{inr\ (pair\ x\ t)}$

¹Произвольный терм, чтобы удовлетворить интерфейсу вариантов.

Элиминатор

$$\mathbf{fold\ xs\ f\ ini} := \mathbf{either\ (K\ ini)\ (\lambda(\mathbf{pair\ x\ t}).\ f\ x\ (\mathbf{fold\ t\ f\ ini}))\ xs}$$

Списки Чёрча

Законы списков

1. $\forall f \text{ ini}. \mathbf{fold} \ \mathbf{nil} \ f \ \text{ini} =_{\beta} \text{ini}$
2. $\forall x \ t \ f \ \text{ini}. \mathbf{fold} \ (\mathbf{cons} \ x \ t) \ f \ \text{ini} =_{\beta} f \ x \ (\mathbf{fold} \ t \ f \ \text{ini})$

Конструкторы

- ▶ По аналогии с числами Чёрча $\mathbf{k} = \lambda s \ z. \underbrace{s \ (s \ (\dots (s \ z) \dots))}_k$
- ▶ $\mathbf{nil} := \lambda s \ z. z$
- ▶ $\mathbf{cons} \ x \ t := \lambda s \ z. s \ x \ (t \ s \ z)$

Элиминатор

- ▶ $\mathbf{fold} \ xs \ f \ \text{ini} := xs \ f \ \text{ini}$
- ▶ $\mathbf{fold} =_{\eta} \mathbf{id}$

Задачи про списки Чёрча

$$[M_0, M_1, \dots, M_{k-1}] := \lambda s \, z. \underbrace{s \, M_0 \, (s \, M_1 \, (\dots (s \, M_{k-1} \, (z) \dots))}_{k} \underbrace{\phantom{s \, M_0 \, (s \, M_1 \, (\dots (s \, M_{k-1} \, (z) \dots))}}_k$$

todo Напишите проверку на пустоту списка

todo Напишите конкатенацию списков

todo Напишите взятие головы списка

isZero $n := n \, (\lambda z'. \text{false}) \, \text{true}$

isEmpty $l := l \, (\lambda x \, z'. \text{false}) \, \text{true}$



plus $n \, m := \lambda s \, z. n \, s \, (m \, s \, z)$

headOrDefault $\text{nil} \, d = d$

headOrDefault $(\text{cons} \, x \, t) \, d = x$



headOrDefault $xs := xs \, \mathbf{K} \, \text{или} \, \mathbf{fst}$

Содержание

1. Ещё метаинформация о курсе
2. Примитивная рекурсия
3. Рекурсия и комбинаторы неподвижной точки
4. Кодирование данных в лямбда-исчислении
 - ▶ Значения-варианты
 - ▶ Списки
5. Энергичность и ленивость
6. Материалы (☁)

Условное выражение как функция

Возьмём какой-нибудь привычный язык программирования. Эквивалентен ли код?

```
if b:
    return f()
else:
    return g()
```

```
def myif(cond, t, e):
    if cond:
        return t
    else:
        return e
myif(b, f(), g())
```

Нет, например:

```
def fac(n):
    if n <= 1:
        return 1
    else:
        return n * fac(n - 1)
```

```
def fac(n):
    return myif(n <= 1,
                1,
                n * fac(n - 1)
                )
```

Что произойдёт? Вычисление разойдётся.

```
fac(3) -> myif(n <= 1, 1, n * fac(n - 1))
        -> myif(n <= 1, 1, n * myif(n - 1 <= 1, 1, (n - 1) * fac(n - 1 - 1)))...
```

Энергичная и ленивая семантики вычисления

Энергичная семантика вычислений

Аргументы функции вычисляются до её вызова (подстановки).

Ленивая семантика вычислений

Сначала происходит вызов функции, аргументы без необходимости не вычисляются.

От энергичной семантики можно перейти к ленивой, как? Лямбды!

```
def fac(n):
    return myif(n <= 1,
               lambda: 1,
               lambda: n * fac(n - 1)
    )

def myif(cond, t, e):
    if cond:
        return t()
    else:
        return e()
```

Содержание

1. Ещё метаинформация о курсе
2. Примитивная рекурсия
3. Рекурсия и комбинаторы неподвижной точки
4. Кодирование данных в лямбда-исчислении
 - ▶ Значения-варианты
 - ▶ Списки
5. Энергичность и ленивость
6. Материалы (☀)

```
struct Node {  
    int value;  
    Node * tail;  
};
```

```
Node * nil = NULL;
```

```
Node * cons(int value, Node * tail) {  
    return new Node { value, tail };  
}
```

Что посмотреть в транспорте

- ▶ Статья: Прimitивно-рекурсивные функции и функция Аккермана

- Klop, J. W. “New fixed point combinators from old.” Reflections on Type Theory, λ -Calculus, and the Mind. Essays dedicated to Henk Barendregt on the occasion of his 60th birthday, Radboud University Nijmegen (2007): 197–211.197–211